Module preservation between HR and LR

Table of contents

Packages 1
Inputs 1
One network per group 2
Preservation 4
Validation 6
Summary 6
Save 6

Do HR and LR share the same co-expression structure? Modules are built inside each group with the settings of `06_Module_Building`, then WGCNA's `modulePreservation()` tests each group's modules in the other group's data (Langfelder et al. 2011). HR/LR only.

Two statistics come back per module. Zsummary combines how dense and how connected the module is in the other group, against permutations: above 10 is strong preservation, 2 to 10 weak to moderate, below 2 none (Langfelder et al. 2011). The median rank orders modules against each other and does not depend on module size; lower is better preserved.

Each group has 7 or 8 people, so each network rests on 20 to 24 biopsies. The test has little power to show a module is missing: a Zsummary above 2 says the module is found again, not that the two networks are the same.

Packages

```
library(here)
library(dplyr)
library(tibble)
library(tidyr)
library(purrr)
library(ggplot2)
library(WGCNA)
source(here("00_Setup", "Shared_functions", "analysis_reporting_helpers.R"))
disableWGCNAThreads()
```

Inputs

```
inputs <- c(
  corrected = "01_Processing/03_Quantification/c_data/DAList_corrected.rds",
  modules = "01_Processing/06_Module_Building/c_data/modules.rds"
)
paths <- map_chr(inputs, here)
stopifnot(file.exists(paths))
corrected <- readRDS(paths[["corrected"]])
modules <- readRDS(paths[["modules"]])
abund <- as.matrix(corrected$data)
```

```
meta <- modules$meta
stopifnot(identical(colnames(abund), meta$sample_id))
bicor_args <- list(corType = "bicor", maxPOutliers = 0.05, pearsonFallback = "individual")
permutations <- 200L
```

One network per group

As in `06_Module_Building`, each group's abundance is centred within person, and people with one biopsy drop out. `apply()` over proteins returns biopsies in rows, the orientation WGCNA expects.

```
centred <- function(subjects) {
  kept <- meta |>
    filter(subject %in% subjects) |>
    add_count(subject) |>
    filter(n > 1)
  apply(abund[, kept$sample_id], 1, \(x) x - ave(x, kept$subject))
}
expr <- map(set_names(c("HR", "LR")), \(group) centred(unique(meta$subject[meta$group ==
group])))
map_int(expr, nrow)
```

```
HR LR
20 24
```

`build_group()` repeats the module construction of `06_Module_Building` on one group's matrix: soft power from `pickSoftThreshold()`, the WGCNA FAQ's signed table as the fallback, and the same `blockwiseModules()` settings and seed.

```
build_group <- function(data) {
  sft <- pickSoftThreshold(data,
    powerVector = 1:20, networkType = "signed",
    corFnc = "bicor", corOptions = list(maxPOutliers = 0.05), verbose = 0
  )
  power <- coalesce(
    sft$powerEstimate,
    c(18L, 16L, 14L, 12L)[findInterval(nrow(data), c(20, 31, 41)) + 1]
  )
  net <- do.call(blockwiseModules, c(
    list(data,
      power = power, networkType = "signed", TOMType = "signed",
      deepSplit = 2, minModuleSize = 30, mergeCutHeight = 0.15,
      pamStage = TRUE, pamRespectsDendro = TRUE, maxBlockSize = 2500,
      numericLabels = FALSE, randomSeed = 54321, verbose = 0
    ),
    bicor_args
  ))
  list(power = power, colours = set_names(net$colors, colnames(data)))
```

```
}
group_modules <- map(expr, build_group)
```

```
   Power SFT.R.sq  slope truncated.R.sq mean.k. median.k. max.k.
1      1  0.15200 12.700          0.962  952.00    953.00  992.0
2      2  0.02080  2.990          0.907  528.00    527.00  591.0
3      3  0.00597  0.879          0.925  316.00    314.00  394.0
4      4  0.00108 -0.231          0.935  200.00    198.00  281.0
5      5  0.05970 -1.140          0.962  133.00    130.00  210.0
6      6  0.12400 -1.270          0.974   91.50     89.20  162.0
7      7  0.16600 -1.210          0.977   65.10     62.90  128.0
8      8  0.22500 -1.260          0.981   47.50     45.30  104.0
9      9  0.31300 -1.360          0.979   35.50     33.30   85.2
10    10  0.36300 -1.370          0.978   27.00     25.10   71.0
11    11  0.46000 -1.500          0.980   20.90     19.10   59.9
12    12  0.52600 -1.570          0.983   16.50     14.70   51.1
13    13  0.59000 -1.620          0.987   13.10     11.60   43.9
14    14  0.65200 -1.680          0.989   10.60      9.13   38.0
15    15  0.69400 -1.760          0.984    8.63      7.28   33.2
16    16  0.73600 -1.800          0.988    7.11      5.86   29.1
17    17  0.76000 -1.830          0.993    5.90      4.76   25.7
18    18  0.78100 -1.830          0.994    4.95      3.91   22.7
19    19  0.80000 -1.840          0.995    4.17      3.21   20.2
20    20  0.82100 -1.850          0.996    3.54      2.66   18.1
   Power SFT.R.sq slope truncated.R.sq mean.k. median.k. max.k.
1      1   0.0296  4.07          0.874  957.00    960.00 1030.0
2      2   0.1440 -4.66          0.939  521.00    518.00  610.0
3      3   0.2470 -3.96          0.951  302.00    299.00  386.0
4      4   0.1320 -2.14          0.929  184.00    181.00  257.0
5      5   0.1060 -1.54          0.935  117.00    115.00  180.0
6      6   0.1190 -1.32          0.921   77.60     74.90  129.0
7      7   0.1500 -1.09          0.920   52.90     50.40   95.8
8      8   0.3880 -1.72          0.959   37.00     34.80   78.1
9      9   0.5760 -2.13          0.972   26.60     24.60   65.5
10    10   0.6800 -2.30          0.935   19.50     17.70   55.8
11    11   0.7660 -2.40          0.939   14.60     12.90   48.1
12    12   0.8080 -2.39          0.921   11.10      9.62   42.0
13    13   0.8630 -2.36          0.940    8.53      7.22   36.9
14    14   0.8930 -2.37          0.938    6.67      5.48   32.7
15    15   0.9110 -2.38          0.937    5.28      4.24   29.1
16    16   0.9040 -2.41          0.923    4.23      3.30   26.1
17    17   0.9110 -2.37          0.916    3.42      2.58   23.5
18    18   0.9290 -2.31          0.931    2.80      2.04   21.3
19    19   0.9280 -2.27          0.921    2.31      1.63   19.3
20    20   0.9430 -2.22          0.937    1.92      1.31   17.6
```

```
map(group_modules, \(x) c(power = x$power, modules = n_distinct(setdiff(x$colours, "grey"))))
```

```
$HR
  power modules
     16      19

$LR
  power modules
     13      15
```

Preservation

Each group is the reference once and the test once. `grey` (unassigned) and `gold` (a random sample WGCNA adds as a baseline) are dropped from the table.

```
preservation <- modulePreservation(
  multiData = map(expr, \(data) list(data = data)),
  multiColor = map(group_modules, "colours"),
  referenceNetworks = 1:2, networkType = "signed",
  corFnc = "bicor", corOptions = "maxPOutliers = 0.05, pearsonFallback = 'individual'",
  nPermutations = permutations, randomSeed = 42, savePermutedStatistics = FALSE,
  verbose = 0
)
```

Each group module is labelled with its best-overlapping module from the full cohort, so the two sets of colour names can be read together.

```
full <- set_names(modules$membership$module, modules$membership$uniprot_id)
best_match <- imap(group_modules, \(built, group) {
  tibble(
    uniprot_id = names(built$colours), group_module = built$colours, full_module =
full[uniprot_id]
  ) |>
    filter(group_module != "grey") |>
    count(group_module, full_module) |>
    mutate(share = n / sum(n), .by = group_module) |>
    slice_max(n, n = 1, by = group_module, with_ties = FALSE) |>
    transmute(reference = group, module = group_module, full_module, full_share = round(share,
2))
}) |>
  list_rbind()
preservation_table <- map(c(HR = "LR", LR = "HR"), \(test) {
  reference <- setdiff(c("HR", "LR"), test)
  pair <- c(paste0("ref.", reference), paste0("inColumnsAlsoPresentIn.", test))
  z <- preservation$preservation$Z[[pair]]
  rank <- preservation$preservation$observed[[pair]]
  tibble(
    reference = reference, test = test, module = rownames(z),
    n_proteins = z$moduleSize, z_summary = z$Zsummary.pres, median_rank = rank$medianRank.pres
  )
}) |>
```

```
  list_rbind() |>
  filter(!module %in% c("grey", "gold")) |>
  left_join(best_match, by = c("reference", "module")) |>
  mutate(preservation = cut(
    z_summary, c(-Inf, 2, 10, Inf),
    labels = c("none", "weak to moderate", "strong")
  ))
count(preservation_table, reference, test, preservation)
```

```
# A tibble: 6 × 4
  reference test  preservation         n
  <chr>     <chr> <fct>            <int>
1 HR        LR    none                 1
2 HR        LR    weak to moderate    13
3 HR        LR    strong               5
4 LR        HR    none                 1
5 LR        HR    weak to moderate    10
6 LR        HR    strong               4
```

```
preservation_table |>
  mutate(direction = paste(reference, "modules tested in", test)) |>
  select(direction, module, n_proteins, full_module,
    `Zsummary` = z_summary, `median rank` = median_rank
  ) |>
  pivot_longer(c(Zsummary, `median rank`), names_to = "statistic") |>
  ggplot(aes(n_proteins, value)) +
  geom_hline(
    data = tibble(statistic = "Zsummary", y = c(2, 10)), aes(yintercept = y),
    linetype = "dashed", colour = "grey55"
  ) +
  geom_point(aes(fill = module), shape = 21, size = 3, colour = "grey30") +
  ggrepel::geom_text_repel(
    aes(label = paste0(module, "\n(", full_module, ")")),
    size = 2.2, lineheight = 0.85, seed = 1, colour = "grey25", min.segment.length = 0
  ) +
  facet_grid(statistic ~ direction, scales = "free_y") +
  scale_fill_identity() +
  scale_x_log10() +
  labs(x = "module size (log scale)", y = NULL) +
  theme_minimal(base_size = 10)
```

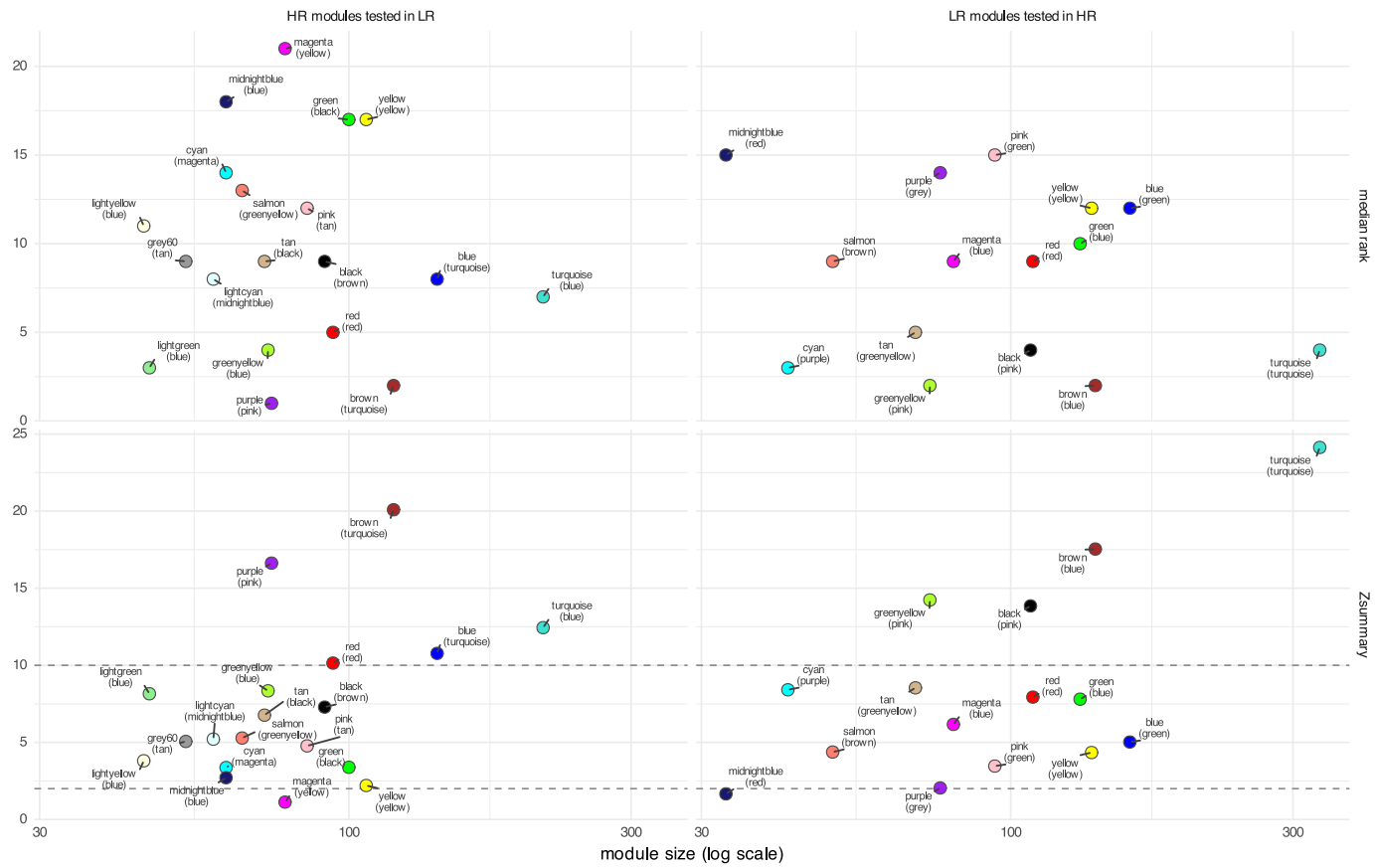


Figure 1: Each group's modules tested in the other; label: best-matching full-cohort module.

Validation

```
stopifnot(
  setequal(unique(preservation_table$reference), c("HR", "LR")),
  !anyNA(preservation_table$z_summary),
  !anyNA(preservation_table$full_module)
)
```

Summary

```
bands <- count(preservation_table, reference, preservation)
band <- \(ref, level) sum(filter(bands, reference == ref, preservation == level)$n)
```

HR modules tested in LR: 5 strongly preserved, 13 weak to moderate, 1 not preserved. LR modules tested in HR: 4 strongly preserved, 10 weak to moderate, 1 not preserved.

Save

```
out <- here("03_Group_comparisons", params$grouping, "c_data")
dir.create(out, recursive = TRUE, showWarnings = FALSE)
```

```
write_workbook(
  list(
    preservation = preservation_table,
    group_networks = tibble(
      group = names(group_modules), n_samples = map_int(expr, nrow),
      power = map_int(group_modules, "power"),
      n_modules = map_int(group_modules, \(x) n_distinct(setdiff(x$colours, "grey")))
    ),
    group_membership = imap(group_modules, \(x, group) {
      tibble(group = group, uniprot_id = names(x$colours), module = x$colours)
    }) |>
      list_rbind()
  ),
  c(
    "Zsummary and median rank of each group's modules tested in the other group.",
    "Samples, soft power and module count of each group's network.",
    "Every protein's module in each group's network."
  ),
  file.path(out, "07_module_preservation.xlsx"), inputs, paths
)
```